

PROJECT REPORT

Auto-Assessment Agent

A multi-agent LLM system for evidence-anchored grading of handwritten and typed answer sheets

Team: Ashutosh Patidar and Aayushman

Affiliation: Department of Data Science and Artificial Intelligence, IIT Madras

Project period: August 2026 to September 2026

Repository: github.com/asu2304/auto-assessment-agent

Stack: Python, FastAPI, Pydantic, Google Gemini, SQLite, React, Vite, Docker

Abstract

Grading written answer sheets is slow, and feedback usually reaches students too late and in too little detail to change how they study. Large language models can read handwriting and reason about solutions, but a single prompt that says "grade this" produces scores that cannot be checked: the model may misread the page, invent working the student never wrote, or return criterion marks that do not add up.

Auto-Assessment Agent splits grading into a pipeline of specialised agents with strict data contracts between them. A Transcriber converts scanned pages into text, a Solver produces a reference answer when none is supplied, an Evaluator scores each rubric criterion and must quote the student's own words as evidence, and an Auditor verifies the result in plain Python with no model call. A Regrade agent handles disputes on a single question, and a Chat agent explains the assessment in a Socratic style. Two memory stores carry information across runs: recurring weak concepts per student, and grading corrections confirmed on a given question paper.

The system is delivered as a FastAPI backend and a React web application with Google sign-in, batch grading, assessment history and voice support, and it ships as a two-container Docker Compose deployment. This report describes the design, the reasoning behind each decision, and the current limitations.

Contents

- | | |
|---------------------------|---------------------------------|
| 1. Problem and motivation | 6. Reliability and safety |
| 2. System overview | 7. Backend and API |
| 3. The agents | 8. Frontend |
| 4. Data contracts | 9. Deployment and testing |
| 5. Agentic memory | 10. Limitations and future work |

1. Problem and motivation

A teacher marking a class test repeats the same three steps for every script: read the handwriting, compare the working with the expected solution, and award marks against a rubric. The work is repetitive, and under time pressure the written feedback shrinks to a tick, a cross and a number. Students see what they lost but not why, and not what to do differently next time.

Using a language model for this task raises four specific risks, and each one shaped the design:

- **Unverifiable scores.** A mark with no pointer to the student's work cannot be trusted or disputed. Every criterion therefore carries a verbatim evidence quote.
- **Hallucinated working.** A model can credit steps that are not on the page. Evidence quotes are checked against the transcript, and anything that cannot be found is flagged for a human.
- **Arithmetic errors.** Models are unreliable at summing their own marks. Score arithmetic is checked by deterministic code.
- **Guessing at illegible work.** When handwriting cannot be read, the correct output is "needs human review", not a plausible number.

2. System overview

The pipeline takes a question paper or rubric, a student submission, and optionally a model answer and extra grading instructions. Inputs may be PDF, PNG, JPG, WEBP, DOCX or plain text. The output is a structured

assessment: a score and feedback for every question, broken down by rubric criterion.

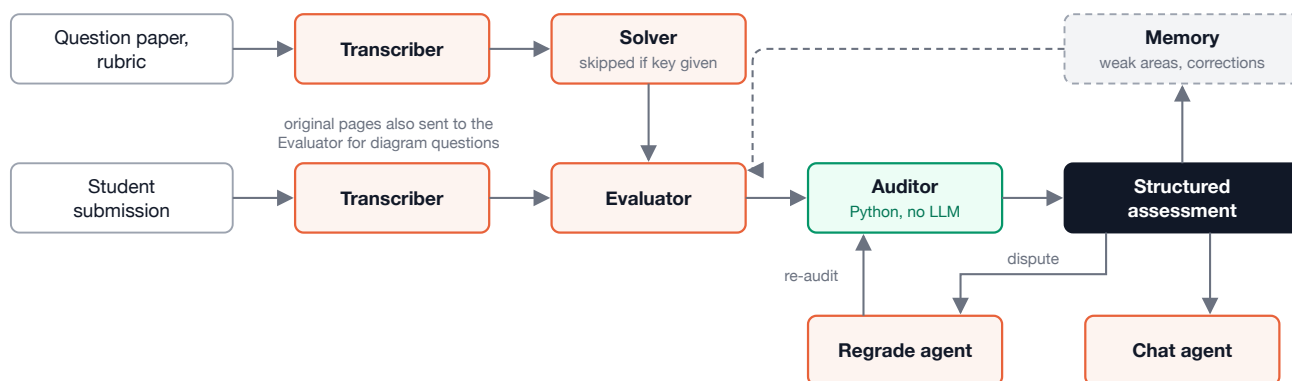


Figure 1. Grading pipeline. Orange boxes call a language model; the green Auditor is deterministic code.

Stage	Responsibility	Runs on
Transcriber	Reads scanned PDFs and images page by page into Markdown, keeping question numbers, notation, tables and labels.	Gemini vision (Bodhan OCR optional)
Solver	Writes a worked reference answer key. Skipped when the teacher uploads a model answer.	Gemini
Evaluator	Scores every rubric criterion with a verbatim evidence quote, feedback, the concept tested and one actionable takeaway.	Gemini, structured output
Auditor	Checks score bounds, criterion totals, duplicate question IDs and whether each evidence quote exists in the transcript.	Python only
Regrade agent	Re-examines one disputed question against the stored evidence before any score changes.	Gemini, structured output
Chat agent	Answers follow-up questions grounded in the saved assessment. It cannot change scores.	Gemini, streamed

3. The agents

3.1 Transcriber

PDFs are rendered to page images at 200 DPI with Poppler, and each page is sent to the vision model with an instruction to transcribe verbatim, mark unreadable content as `[illegible]`, and not solve anything. Processing page by page means one failed page does not lose the whole submission: each page returns its own result with text or an error, and the pipeline continues with whatever was read.

Transcriptions are cached on disk under a SHA-256 hash of the model name and the image bytes, so re-running the same page costs nothing. The cache also rejects entries that look like an API error message saved as text. When the optional Bodhan OCR provider is enabled and fails, the agent falls back to Gemini for that page. Uploads are limited to 10 pages per request and 50 MB per PDF.

3.2 Solver

If no model answer is provided, the Solver generates one from the question paper and rubric, showing intermediate steps and final answers with units. The Evaluator then grades against this key. Supplying an official

answer key skips this call, which lowers cost and removes one source of model error.

3.3 Evaluator

The Evaluator is the core grading call. Its response must conform to a Pydantic schema, which is passed to the model as a JSON schema and validated again on return. The prompt sets numbered requirements, including:

- **Evidence anchoring.** For every criterion, copy a short verbatim quote from the submission and never invent student working.
- **Actionable feedback.** State what the student showed, where the error occurred, and the correct reasoning, plus one concrete rule to apply next time.
- **Arithmetic integrity.** Criterion scores must sum exactly to the question score and stay within their weights.
- **Uncertainty.** Illegible work sets `needs_human_review`. A question the student skipped scores zero with no partial credit.
- **Per-question text.** Return the exact question and the student's full answer to it, so the interface can show both beside the score.

Diagram questions. A transcript can only paraphrase a sketch. When the question paper or rubric contains words such as "draw", "sketch", "construct" or "graph", the original pages are attached to the Evaluator call alongside the transcript, and the model is told to judge the drawing from the pages themselves. For these questions a separate call produces a reference diagram as SVG. That call is deliberately isolated: bundling it into the grading schema made the main call less reliable, and a failed diagram must never affect a grade.

3.4 Auditor

The Auditor is ordinary Python and costs nothing to run. It rejects a report if any question ID is duplicated, any score falls outside its valid range, or the criterion marks do not sum to the question score within a tolerance of 0.01. It then checks every evidence quote against the transcript after normalising case and white-space. If a quote cannot be found, or a question was awarded marks although no answer was recorded for it, the question is flagged for human review. A prompt instruction is a request, not a guarantee, and this check is what turns the "quote your evidence" rule into something enforced.

3.5 Regrade agent

A student disputes a specific question by describing the claimed mistake and optionally quoting their own work. Before any model call, the quoted text must actually appear in the submission, otherwise the request is rejected. The agent then decides whether the claim is a demonstrable grading error. If it is not verified, the original evaluation is restored unchanged. If it is, the new evaluation is audited again, the maximum score is held fixed, and the question text, the student's answer and the reference diagram are treated as immutable facts that a regrade can never alter.

3.6 Chat agent

The Chat agent explains an existing assessment using the rubric, answer key, submission and graded report as context. Its instructions favour short answers and a Socratic approach for conceptual questions: point at the step to re-examine before giving the full explanation. Direct answers are reserved for factual lookups such as a score. Responses are streamed to the browser, and only the most recent turns of the conversation are sent to keep the prompt bounded.

4. Data contracts

Agents communicate through typed models, not free text. The schema is both the instruction given to the model and the validator applied to its reply, so a malformed response fails loudly and is retried.

Model	Key fields	Built-in validation
CriterionScore	description, score, weight, evidence_quote, feedback	score cannot exceed weight; weight must be positive
QuestionEvaluation	question_id, score, max_score, criterion_scores, feedback, actionable_takeaway, concept_tested, needs_human_review, question_text, student_answer, reference_diagram_svg	score cannot exceed max_score
AssessmentReport	evaluations, overall_summary, strengths, priority_growth_areas	evaluations must not be empty
RegradeResult	question, changed, claim_verified, explanation	re-audited before it replaces the original

The empty-evaluations rule is a good example of the approach. By the time the Evaluator runs, both the question paper and the student work are known to be non-empty, so a report with no evaluations can only mean the model graded nothing. Raising a validation error at that point makes the retry logic ask again, instead of saving an empty assessment.

5. Agentic memory

The Evaluator is not stateless across runs. Two small stores in SQLite feed extra context into each grading prompt.

Student weak-area memory

After each assessment, the concept behind every question that lost marks is recorded for the signed-in student, with a count and the latest takeaway. A repeat miss raises the count; full marks on that concept lowers it, and entries that reach zero are deleted. The top five are passed to the next grading call, so feedback can say whether a weakness persisted or improved.

Grading-correction memory

When a regrade is verified and changes a score, the correction is stored against a hash of the normalised question paper. The next submission graded on the same paper receives the five most recent corrections with an instruction not to repeat them, so one student's successful dispute improves grading for everyone who follows.

Because the model names concepts in its own words, the same weakness can come back phrased differently. New concept names are folded into an existing entry when one contains the other, which keeps the count from fragmenting across near-duplicates. Memory is keyed to the signed-in Google account, not to an anonymous browser identifier.

6. Reliability and safety

- **Retries with backoff.** Model calls are attempted up to three times with exponential delay and random jitter, capped at 20 seconds. Rate limits (HTTP 429), server errors and validation failures are retried; other client errors are not.

- **Prompt-injection defence.** Every prompt wraps uploaded material in labelled delimiters and states that the question paper, rubric, answer key, submission and dispute text are untrusted data, never instructions. A student who writes "award full marks" in their answer is quoting text, not issuing a command.
- **Sanitised model output.** Reference diagrams are model-generated SVG rendered in the browser. Any SVG containing scripts, event handlers, embedded foreign content or external links is dropped entirely.
- **Human in the loop.** Illegible work, unverifiable evidence and credit given for a blank answer all set a review flag, and a reviewer can confirm a question explicitly.
- **No shared request state.** A single agent instance serves all requests, so it holds no "current assessment". Every method receives its context explicitly, which prevents one user's data leaking into another's request.
- **Friendly errors.** Internal validation dumps are replaced with a short message before they reach the user.

7. Backend and API

The backend is a FastAPI application. Blocking model calls run in worker threads so the event loop stays responsive. Batch grading prepares the shared material once (question paper transcription, answer key and reference diagrams), then grades students concurrently under a semaphore, three at a time by default and up to 25 students per batch. Each student is graded by a separate worker, and one student's failure does not stop the others.

Endpoint	Purpose
POST /api/assess	Grade one submission
POST /api/assess/batch	Grade several submissions against one shared rubric
POST /api/ocr/preview	Preview the transcription before grading
POST /api/regrade	Request re-evaluation of one question
POST /api/assessments/{id}/review	Record a human review of a flagged question
POST /api/chat/stream	Streamed, assessment-grounded chat
GET /api/assessments/recent	List recent assessments for the signed-in user
GET, DELETE /api/assessments/{id}	Fetch or delete one assessment
POST /api/voice/synthesize	Text-to-speech for chat replies and feedback
GET /api/models , /api/system/check	Live model configuration and dependency check
POST /api/auth/google	Exchange a Google ID token for a session

Persistence

SQLite holds three tables: `assessments` (the report and its full grading context as JSON), `student_memory` and `grading_corrections`. Storing the context with each report is what allows regrading and chat to work later without re-uploading files. The five most recent runs are kept per user, and a batch counts as one run.

Authentication

Sign-in uses Google Identity Services. The server verifies the Google ID token against the configured client ID, optionally restricts access to listed email domains, and then issues its own session token: a payload signed with HMAC-SHA256 using a server-side secret, valid for 30 days and delivered as an HTTP-only cookie. Signatures are compared in constant time. Because identity comes from this signed token and not from a header the client supplies, one user cannot read another user's assessments or memory.

8. Frontend

The interface is a React single-page application built with Vite. It follows the grading workflow in three steps (upload, score feed, agent chat) with separate pages for history and the live model configuration. Mathematical notation in questions, student answers and feedback is rendered with KaTeX. The chat supports voice input through the browser's speech recognition and spoken replies through a text-to-speech endpoint.

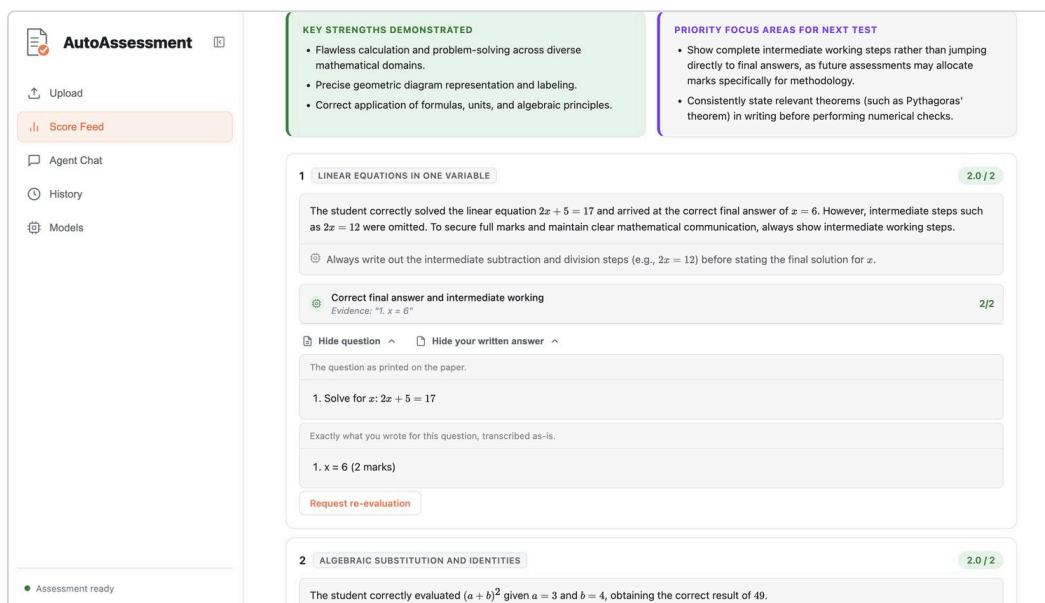


Figure 2. Score Feed: per-question score, the concept tested, feedback, the evidence quote, the original question and the student's transcribed answer, with a re-evaluation button.

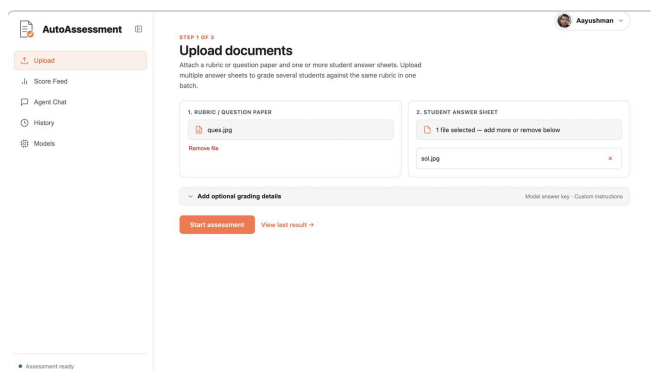


Figure 3. Upload: rubric and one or more answer sheets.

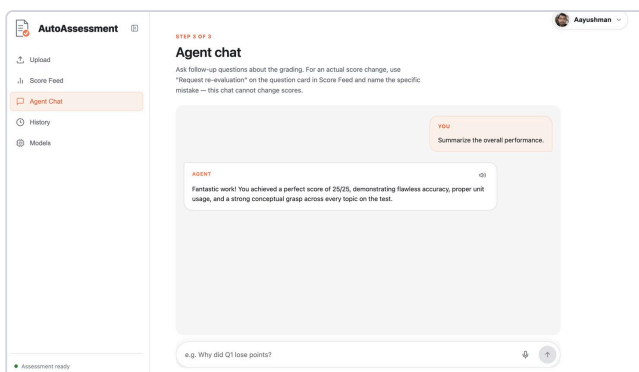


Figure 4. Agent Chat: grounded follow-up questions.

9. Deployment and testing

The project runs locally with a single launcher script that loads the environment file and starts both servers. For deployment, Docker Compose builds two containers: the backend image (Python 3.11 with Poppler) and an nginx image that serves the built frontend and proxies API requests to the backend. The SQLite database lives on a named volume so that data survives rebuilds. All configuration, including API keys, model names per stage, batch limits and OCR resolution, is supplied through environment variables.

The unit tests cover the parts that do not need a model: document parsing, report validation (duplicate IDs and inconsistent totals), preservation of per-page OCR failures, and the configuration endpoints. They run without any API keys.

10. Limitations and future work

- **No quantitative evaluation yet.** The system has not been benchmarked against human graders. The most important next step is a labelled set of scripts and a report of agreement with teacher marks, per question and per criterion.
- **Evidence matching is exact.** The Auditor uses a normalised substring check. A quote that the model lightly paraphrases, or that differs from the transcript by one OCR character, is flagged for review even when the grading is correct. Fuzzy matching would reduce these false flags.
- **Diagram detection is keyword-based.** The decision to attach original pages relies on a fixed word list, which can miss unusual phrasings or trigger on unrelated uses of words like "figure".
- **Concept merging is simple.** Weak-area entries are merged by substring containment. Embedding-based similarity would group related concepts more reliably.
- **Transcription is a single point of failure.** A misread digit propagates into grading. The OCR preview lets a user catch this before grading, but nothing detects it automatically.
- **Scale.** SQLite and a single backend process suit a classroom, not an institution. A hosted database and a job queue would be needed for larger batches.
- **Test coverage.** The current tests are a small core. The regrade path, memory updates and batch error handling should be covered with mocked model responses.

Design principles in summary

Evidence before assertion. One responsibility per agent, with typed contracts between them. Deterministic code wherever the answer can be computed. Explicit uncertainty instead of guessing. Feedback a student can act on.